

信息检索实验报告

基于 TF-IDF 与 BM25 的技能检索系统

acknowledge 项目实验记录

2026 年 5 月 29 日

摘要

这次实验围绕 skills.sh 上的 1000 条 AI 技能描述做了一个离线检索系统。数据先由 paqu.py 抓下来，再把标题、描述、仓库信息和外置 SKILL.md 统一规整到同一份数据集里。检索部分没有走重模型路线，而是把 TF-IDF 余弦相似度、BM25 和一组后置 boost 规则叠在一起，目标很直接：查询里既有英文短词，也有中文任务描述，结果还得能解释为什么排成这样。

按当前仓库里的真实运行结果，索引覆盖 1000 篇文档、15543 个 term，索引文件大小约 9.10 MB。对 core_relevance.json 中 42 条查询做评测时，纯 TF-IDF 的 MRR@5 最好，为 0.9107；混合检索的 P@5 和 nDCG@5 稍高，分别达到 0.2810 和 0.8988。固定权重扫描也说明 BM25 更像补分项，权重压在 0.02 到 0.05 一段时效果最稳。

1 引言

这个项目面对的问题不复杂：给一条任务需求，尽量快地从技能库里找出合适的 skill。难点不在规模，而在文本形态很杂。查询既可能是 API design 这种英文短语，也可能是“前端页面设计”“向量搜索 Qdrant”这种中文或中英混合表达。只靠标题精确匹配会漏召回，只靠模糊匹配又容易把一堆相关但不对题的 skill 推上来。

数据来自 skills.sh 的 1000 条 skill 记录，主文件是 data/skills_data.json。每条记录至少包含 skill_name、owner、repo、description、category、detail_url、repo_url、first_seen、weekly_installs_num、github_stars_num 等字段；较长的 SKILL.md 文本被外置到 data/skill_md/ 目录，主数据里只保留路径。当前快照里一共有 917 条记录带外置 SKILL.md，203 条记录缺 description，所以检索不能只盯着一个字段看。

这次实验的目标有三件事：第一，把爬虫、清洗、索引这条链路说明白；第二，把 TF-IDF、BM25、动态融合和 boost 规则写清楚，不只报几个分数；第三，用真实评测结果看一下哪一部分真的在起作用，哪一部分只是看起来很合理。

2 数据获取与预处理

2.1 爬虫与落盘流程

paqu.py 做的是一条偏工程化的抓取链路。首页靠滚动收集详情页 URL，详情页再用 curl/urllib 拉 HTML，之后从 SSR 文本和 `__next_f.push` 片段里拆出字段。如果页面里有 Show more，脚本会退到 Selenium，把完整 SKILL.md 展开后再提取。抓取过程中所有中间结果都会写进 checkpoint，失败 URL 也会单独记录，后面可以重试。整个流程见图 1。

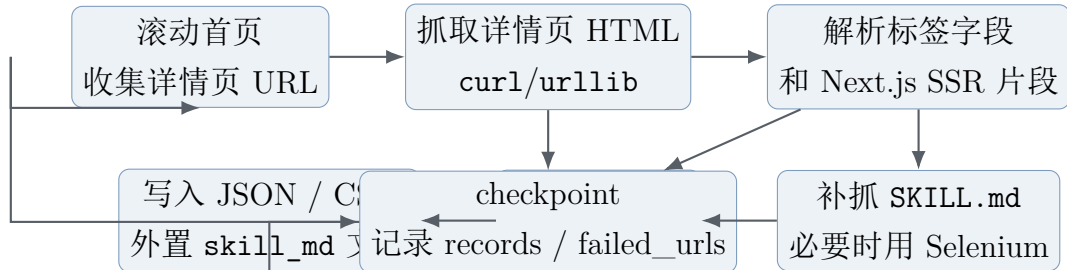


图 1: 数据抓取与落盘流程: URL 先由首页滚动收集, 详情页字段与 SKILL.md 再分层解析, checkpoint 贯穿整个过程, 用来承接失败重试和断点续跑。

2.2 数据字段与完整性

表 1 列了检索阶段最常用的字段。这里我没有把所有原始键都摊开, 只保留对排序和结果解释最有用的部分。

表 1: 检索系统使用的主要字段与当前数据完整性统计

字段名	含义	非空条数
skill_name	技能名, 标题 boost 的核心字段	1000
description	简介文本, 检索主语料之一	797
category	规则推断或原始类别, 用于轻量补分	1000
repo_url	GitHub 仓库链接, 辅助说明结果来源	1000
weekly_installs_num	周安装量, 参与热度修正	992
github_stars_num	GitHub star 数, 参与热度修正	984
skill_md_text_path	外置 SKILL.md 纯文本路径	917
skill_md_raw_text_path	外置原始文本路径	917

这批数据有两个特点。第一, 文本主体几乎全是英文, 标题和描述同时含 CJK 的记录只有 5 条, 说明“中英混合”更多出现在查询侧, 不是文档侧。第二, SKILL.md 的补齐价值很高。主数据只有 797 条带 description, 但有 917 条能读到外置技能文档, 后面做检索时如果只用标题和简介, 很多长文档里的关键术语根本进不了索引。

2.3 文本预处理

IR 子系统的预处理在 `ir_system/src/skills_ir/text.py`。它没有接第三方分词器，而是自己做了一套轻量规则：

- 英文 token 用正则抓取后，再生成若干变体，例如把连字符和下划线拆开，把复数还原成词干。
- 中文片段同时保留整段、单字和双字切片，用来弥补没有中文分词器时的召回损失。
- 文档建模时按字段权重累计词频，标题权重 4.0，描述权重 3.0，类别权重 2.5，仓库 owner / repo 各 1.2。
- 查询阶段还会查配置里的扩展词典，例如“前端”会扩成 `frontend/ui/ux/react/vue/css`，“向量数据库”会扩成 `vector/database/qdrant/embedding/retrieval`。

这种做法不精致，但对当前数据集足够实用。它的优点是可控、可解释、重建成本低；缺点也很明显，中文 token 会被切得比较碎，所以后面混合检索里我会专门把中文查询的 BM25 权重压低。

3 核心算法

3.1 TF-IDF 与余弦相似度

文档先被映射到带字段权重的词频向量。设 $tf_{t,d}$ 表示 term t 在文档 d 里的加权词频， N 为文档总数， df_t 为 term 的文档频次，那么代码里采用的逆文档频率是

$$\text{idf}(t) = \log \frac{N+1}{df_t+1} + 1. \quad (1)$$

文档向量中的 term 权重写成

$$w_{t,d} = (1 + \log tf_{t,d}) \cdot \text{idf}(t). \quad (2)$$

查询向量沿用同一套权重定义，最后的余弦相似度为

$$s_{\cos}(d, q) = \frac{\sum_{t \in q \cap d} w_{t,q} w_{t,d}}{\|q\|_2 \|d\|_2}. \quad (3)$$

这里选对数词频而不是原始词频，主要是想压一压长 `SKILL.md` 里高频重复术语的影响。当前数据里长文档不少，直接堆原始词频会把“模板化重复描述”推得太高。

3.2 BM25

BM25 部分采用 Robertson 和 Zaragoza 总结过的标准写法 [Robertson et al.(2009)Robertson and Za] 对查询 q 和文档 d ，得分为

$$s_{\text{bm25}}(d, q) = \sum_{t \in q} \text{idf}_{\text{bm25}}(t) \cdot \frac{tf_{t,d}(k_1 + 1)}{tf_{t,d} + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}, \quad (4)$$

其中

$$\text{idf}_{\text{bm25}}(t) = \log \left(\frac{N - df_t + 0.5}{df_t + 0.5} + 1 \right). \quad (5)$$

项目里固定 $k_1 = 1.5$ 、 $b = 0.75$ 。这个选择比较保守： k_1 留出一定词频增益，但不会把重复堆词推得太夸张； $b = 0.75$ 则保留了常见的长度归一化强度。因为文档平均长度已经达到 331.83 个 token，长度归一化如果太弱，长 SKILL.md 会吃掉不少短 skill 的位置。

3.3 混合融合与动态权重

BM25 原始分数和余弦分数不在同一量纲上，代码先做最大值归一化：

$$\tilde{s}_{\text{bm25}}(d, q) = \frac{s_{\text{bm25}}(d, q)}{\max_{d'} s_{\text{bm25}}(d', q)}. \quad (6)$$

然后按式 (7) 融合：

$$s_{\text{hyb}}(d, q) = (1 - \alpha_q) s_{\text{cos}}(d, q) + \alpha_q \tilde{s}_{\text{bm25}}(d, q). \quad (7)$$

这里最关键的是 α_q 不是固定常数，而是按查询形态动态设定：

$$\alpha_q = \begin{cases} 0.02, & q \text{ 含 CJK 字符,} \\ 0.08, & q \text{ 不含 CJK 且有效 token 数} \leq 2, \\ 0.10, & 3 \leq \text{token 数} \leq 4, \\ 0.12, & 5 \leq \text{token 数} \leq 8, \\ 0.14, & \text{token 数} > 8. \end{cases} \quad (8)$$

这样设计的原因很直接。当前文档侧几乎全英文，中文查询主要靠扩展词典把语义桥接到英文术语上。如果这时把 BM25 权重给高了，细碎的中文 token 反而容易制造噪声；英文短查询则更适合让 BM25 补一点“词面直击”的分。

3.4 后置 boost 规则

只靠式 (7) 还不够。很多用户更在乎“标题是不是直接对上了”“这个 skill 是不是很热”，这些东西单纯的向量分数感知得不强，所以代码又做了一层加法 boost：

$$s(d, q) = s_{\text{hyb}}(d, q) + \Delta_{\text{title}} + \Delta_{\text{desc}} + \Delta_{\text{category}} + \Delta_{\text{coverage}} + \Delta_{\text{phrase}} + \Delta_{\text{pop}}. \quad (9)$$

各项的具体含义如下。

- 查询完整串落在标题中时，+0.25；落在描述中时，+0.12。
- 查询 token 命中 category 时，+0.05。
- 对短查询或中文查询，如果某个 token 与标题完全相同，再额外 +0.22。
- 标题命中的 token 每个加 0.12，上限 0.36；description 追加命中的 token 每个加 0.02，上限 0.12。
- token 覆盖率按比例折算，最高再给 0.30。
- 配置文件里还写了若干领域短语规则，例如 Qdrant、PDF、video translation 等。
- 热度修正采用

$$\Delta_{\text{pop}} = 0.006 \log(1 + \text{installs}) + 0.005 \log(1 + \text{stars}). \quad (10)$$

我比较认可这里的取舍。boost 不是拿来替代主排序的，而是补统计模型不擅长表达的那一点偏好。真正决定底盘的还是 TF-IDF 和 BM25；热度项最多只是把两个相关度接近的结果再拉开一点。

3.5 倒排索引结构

索引最终会写到 `ir_system/runtime/skills_ir_index.json`。表 2 给出了主要字段和当前规模。

表 2: IR 倒排索引的核心字段与当前规模

字段	含义	当前规模
documents	原始文档数组，保留排序后回填所需字段	1000
postings	TF-IDF 倒排表: $\text{term} \rightarrow \{\text{doc: weight}\}$	15543 个 term
idf	余弦空间的 IDF 表	15543
doc_norms	文档向量二范数	1000
bm25_idf	BM25 的 IDF 表	15543
bm25_tf	BM25 词频表: $\text{term} \rightarrow \{\text{doc: tf}\}$	84696 个 posting 对
doc_lengths	文档长度表	1000
avg_doc_length	语料平均长度	331.83

索引文件总大小约 9.10 MB，84696 个 posting 对平均下来每个 term 连接 5.45 篇文档，说明词表还是挺稀疏的。对这种规模的数据，JSON 落盘虽然不算最省空间，但调试时直接能看，性价比很高。

3.6 检索流程伪代码

算法 1 把实际搜索流程压缩成了几个关键步骤。代码里还有结果去重、snippet 生成等细节，这里先不展开。

算法 1: 混合检索主流程

输入: 查询 q , 返回条数 k

输出: 排序后的结果列表 R

```

1  $T \leftarrow \text{TOKENIZE}(q)$ 
2  $T' \leftarrow \text{EXPANDQUERY}(T)$ 
3 if  $T'$  为空 then
4   return []
5  $Q \leftarrow \text{BUILDQUERYWEIGHTS}(T')$  using Eq. (2)
6  $S_{\text{cos}} \leftarrow \text{SPARSECOSINEACCUMULATE}(Q, \text{postings})$ 
7  $S_{\text{bm25}} \leftarrow \text{BM25SCORE}(T', \text{bm25\_tf}, \text{doc\_lengths})$ 
8  $\alpha_q \leftarrow \text{CHOOSEALPHA}(q, T')$  using Eq. (8)
9 foreach 候选文档  $d$  do
10    $\tilde{s}_{\text{bm25}}(d, q) \leftarrow S_{\text{bm25}}(d, q) / \max S_{\text{bm25}}$ 
11    $s_{\text{hyb}}(d, q) \leftarrow (1 - \alpha_q)S_{\text{cos}}(d, q) + \alpha_q\tilde{s}_{\text{bm25}}(d, q)$ 
12    $s(d, q) \leftarrow \text{APPLYBOOSTS}(d, q, s_{\text{hyb}})$ 
13 按  $s(d, q)$  降序排序
14 按标准化标题去重, 保留每个 skill name 的最高分结果
15 return top- $k$  结果

```

4 实验设计与过程

4.1 实验环境

实验在 Windows 10 (10.0.26100) 上完成, conda 环境名是 yolov82, Python 解释器为 D:\anaconda\envs\yolov82\python.exe, 版本 3.11.5。LaTeX 编译器是 XeTeX 0.999996, 来自 TeX Live 2024。检索部分本身不依赖 GPU。

4.2 评测集与指标

主评测集使用 ir_system/eval/core_relevance.json, 共 42 条查询, 其中英文 24 条、中文 6 条、mixed 12 条。CLI 代码里原生实现了 Hit@K、Top1 Accuracy、MRR@K、Recall@K、Precision@K; 为了把排序位置区分得更细, 我额外补算了 nDCG@5。这里的折扣累计增益采用二值相关性:

$$\text{nDCG}@k = \frac{\sum_{i=1}^k \frac{rel_i}{\log_2(i+1)}}{\sum_{i=1}^{\min(k, |G|)} \frac{1}{\log_2(i+1)}}, \quad (11)$$

其中 G 为当前查询的相关 skill 集合。

除了主评测集，我还看了 `multilingual.json`，主要用来观察中文和 `mixed` 查询在不同模式下的差别。它不是主表的来源，但对解释动态 α 很有帮助。

4.3 运行命令

实验里实际用到的命令如下：

Listing 1: IR 子系统的主要运行命令

```
python ir_system/skills_ir_system.py build-index
python ir_system/skills_ir_system.py evaluate --eval-set core_relevance.json
python ir_system/skills_ir_system.py compare-modes --eval-set core_relevance.json
python ir_system/skills_ir_system.py search "Qdrant_vector_search" --top-k 5
```

索引写到 `ir_system/runtime/skills_ir_index.json`，评测结果和模式对比分别写到 `metrics_report.json` 与 `comparison_report.json`。

5 实验结果与对比分析

5.1 三种检索模式对比

表 3 汇总了 `core_relevance.json` 上的主要结果。Hit@5 三种模式都是 1.0，说明相关结果至少都能进前五；真正拉开差别的是排序位置和前五名里相关结果的占比。

表 3: 三种检索模式在 `core_relevance.json` 上的表现对比

模式	P@5	nDCG@5	MRR@5	Hit@5
TF-IDF	0.2762	0.8962	0.9107	1.0000
BM25	0.2714	0.8671	0.8996	1.0000
Hybrid	0.2810	0.8988	0.9087	1.0000

这个结果挺符合直觉。纯 TF-IDF 的 MRR@5 最高，说明在把最相关结果顶到第 1 位这件事上，向量空间模型还是主力。BM25 单独跑时不算差，但在这批数据上没有超过 TF-IDF。混合检索的作用更像“保守补分”：它没有把 MRR 再抬高，但把 P@5 和 nDCG@5 稍微往前推了一点，前五名整体更整齐。

5.2 BM25 权重扫描

为了看清楚 α 该压在什么区间，我把 BM25 权重固定后重新扫了一遍。图 2 给了 P@5 和 nDCG@5 随 α 的变化。

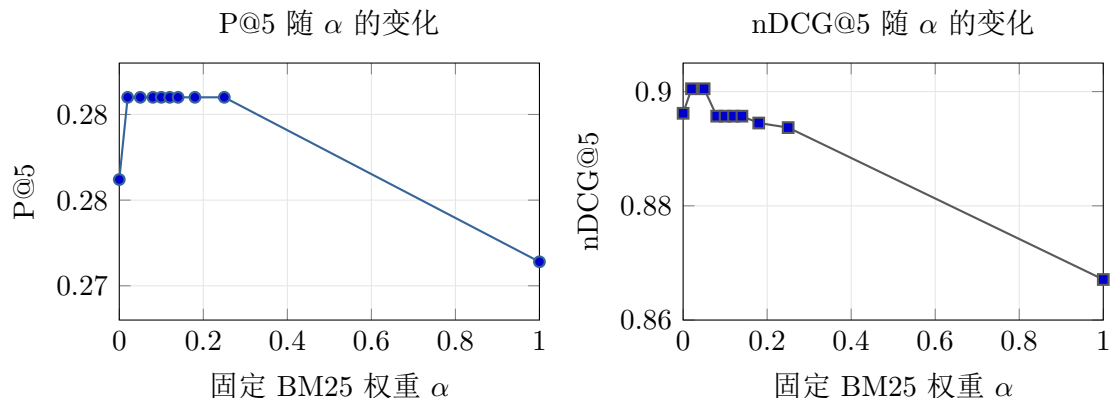


图 2: 固定 BM25 权重扫描结果: 在当前评测集上, $\alpha = 0.02 \sim 0.05$ 一段最稳; 继续加大 BM25 占比后, 排序质量开始缓慢下滑, 纯 BM25 退化最明显。

这张图说明了两件事。第一, BM25 确实能帮一点忙, 但只适合少量掺进去。第二, 当前代码里那套“中文/短查询权重更低, 长英文查询再稍微提高”的策略不是拍脑袋定的, 它刚好落在最稳的一段附近。

5.3 中文、英文与 mixed 查询差异

表 4 是 hybrid 模式按查询类型拆开的结果。英文查询的平均 α 为 0.0942, 中文和 mixed 查询都压到 0.02。

表 4: Hybrid 模式按查询类型拆开的表现与平均 BM25 权重

查询类型	数量	平均 α	P@5	nDCG@5	MRR@5
English	24	0.0942	0.2750	0.9174	0.9410
Chinese	6	0.0200	0.2667	0.9385	0.9167
Mixed	12	0.0200	0.3000	0.8419	0.8403

这里最有意思的是中文查询。虽然它们的 BM25 权重最低, nDCG@5 反而不差。这和数分布有关: 文档主体几乎全英文, 中文查询能命中, 主要靠扩展词典把“前端”“结构化数据”“向量检索”映到对应英文术语上。这一步成功以后, TF-IDF 的向量相似度已经能把主相关结果推上来; BM25 再往上加, 只会让零散中文 token 和局部词面匹配掺进更多噪声。

我又用 multilingual.json 做了交叉检查。那套评测里中文查询只有 4 条, 纯 TF-IDF 和 hybrid 的 nDCG@5 都是 0.6577, BM25 则掉到 0.5886。这和式 (8) 的方向是一致的: 中文部分先别让 BM25 说太多话。

5.4 失败案例

虽然三种模式的 Hit@5 都是 100%, 但还有几个典型问题值得记一下。

- `copywriting for marketing`: 相关 skill 在前五里, 但 `marketing-skills-collection` 和 `marketing-automation` 这类泛化程度高的结果会抢到更前面, 说明领域词过宽时, 主题相关性和“任务是否对题”还没有完全拆开。
- `browser automation`: 相关结果有多个, `actionbook` 常常排不进更靠前的位置, 说明标题和描述里直说“`browser automation`”的 skill 更占便宜, 隐式相关项吃亏。
- `SEO audit:seo-audit` 往往能到第 1, 但另一个期望结果 `domain-authority-auditor` 不够靠前, 暴露出多相关文档场景下的覆盖问题。
- `SQL queries: supabase-postgres-best-practices、database-optimizer` 这类近邻文档有时会把 `sql-queries` 挤到第 3 或第 4, 说明“强相关概念邻居”对纯词法模型还是个老问题。

6 总结与展望

这套 IR 系统跑下来, 最大的感受是“简单模型不等于简单处理”。真正有用的不是单独的 TF-IDF 或 BM25, 而是整套围绕数据特征做的小修补: 字段加权、查询扩展、动态 α 、标题和热度 boost。当前数据上, TF-IDF 还是主骨架; BM25 更适合做轻量补分, 权重大了反而坏事。

局限也很清楚。第一, 文档侧英文占绝对多数, 中文检索基本靠手工扩展词典兜着。第二, 评测脚本里还没有原生 `nDCG`, 想看排序细节得另外补算。第三, 规则 boost 现在是手调常数, 还没有学到更细的排序偏好。后面如果继续做, 我会优先补三件事: 扩中文扩展词典、把 `nDCG` 正式纳入 CLI 评测、再加一层轻量 reranker, 把“概念相关但不是答案”的结果往后压一点。

参考文献

[Robertson et al.(2009)Robertson and Zaragoza] ROBERTSON S, ZARAGOZA H. The probabilistic relevance framework: Bm25 and beyond[J]. Foundations and Trends in Information Retrieval, 2009, 3(4): 333-389.